



Compiladores

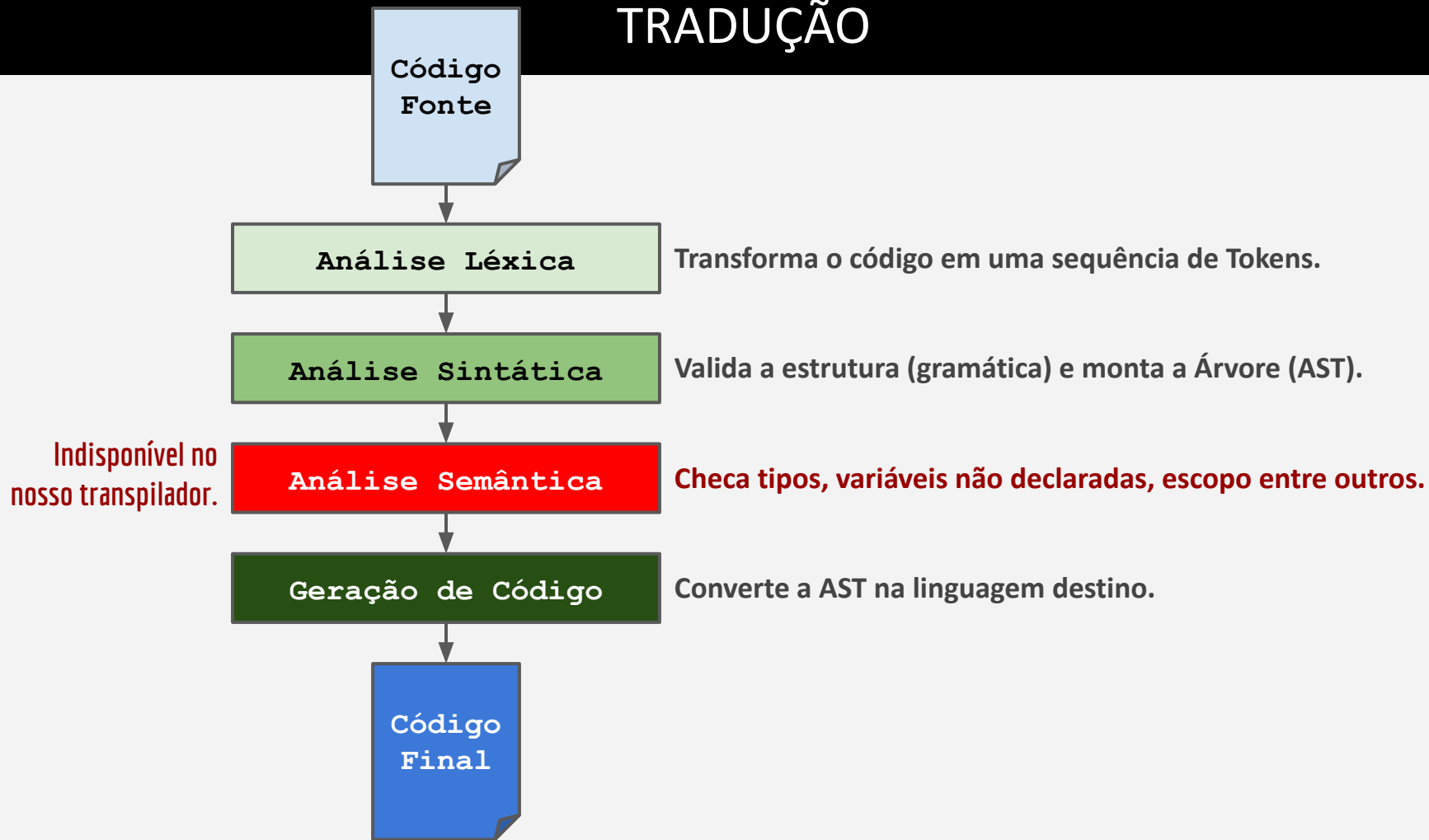
Material 6

Geração do Código

“Conversão da AST na linguagem destino.”

Prof. Roger Rocha Ferreira

TRADUÇÃO





BNF

```
<programa> ::= <instrucoes> <retorno>
<instrucoes> ::= <instrucao> | <instrucao> <instrucoes>
<instrucao> ::= <atribuicao>
                | <saida>
                | <entrada>
                | <condicional>
                | <loop>
<atribuicao> ::= NOME IGUAL <expressao>
<saida> ::= ESCREVA ABRE_PAR <expressao> FECHA_PAR
<entrada> ::= NOME IGUAL LEIA ABRE_PAR TEXTO FECHA_PAR
<condicional> ::= SE <condicao> ABRE_CHAVE <instrucoes> FECHA_CHAVE
                | SE <condicao> ABRE_CHAVE <instrucoes> FECHA_CHAVE SENAO ABRE_CHAVE
<instrucoes> FECHA_CHAVE
<loop> ::= ENQUANTO <condicao> ABRE_CHAVE <instrucoes> FECHA_CHAVE
<condicao> ::= <expressao> <operador_relacional> <expressao>
              | NEGACAO <condicao>
<operador_relacional> ::= IGUALDADE | MENOR | MAIOR
<expressao> ::= <termo>
              | <expressao> SOMA <termo>
              | <expressao> SUB <termo>
```

```
<termo> ::= <fator>
          | <termo> MUL <fator>
          | <termo> DIV <fator>
<fator> ::= NUMERO
          | NOME
          | ABRE_PAR <expressao> FECHA_PAR
<retorno> ::= RETORNO NUMERO
```

Lexer / Analisador Léxico / Tokenizer

Transformar o código-fonte bruto em uma lista de **tokens** significativos.

Produto:

Um arranjo de textos compreendendo as palavras e pontuações da pseudo-linguagem.

Função:

Separar as instruções de linguagem eliminando os espaços e caracteres inúteis.

As Funções Principais do Lexer

Agrupamento de Caracteres (Lexemas)

Lê o texto caractere por caractere e agrupa padrões conhecidos.

Categorização em Tokens

Associa cada valor lido (lexema) a um tipo/categoria que o Parser possa entender.

Filtragem de "Ruído"

Remove elementos que não afetam a lógica do programa, como espaços em branco desnecessários, quebras de linha e comentários.

Detecção de Erros Léxicos

Identifica caracteres inválidos que não pertencem ao alfabeto da linguagem.

Parser / Analisador Sintático

Verifica se a sequência de *tokens* gerada pelo **Lexer** segue as regras gramaticais da linguagem.

Produto:

Árvore Sintática Abstrata - AST, a estrutura hierárquica do programa.

Função:

compreender as "frases e orações" da pseudo-linguagem.

As 3 Funções Principais do Parser

Análise Sintática (Validação de Gramática)

Garante que os tokens estejam na ordem correta.

Construção da AST (Árvore Sintática Abstrata)

Transforma uma lista linear de tokens em um nó estruturado em árvore que organiza a hierarquia do código.

Mapeamento de Precedência de Operadores

Garante a regra da matemática ao construir os nós provendo a precedência aritmética posicionando valores prioritários em nós mais fundos.



Parser

```
class Parser:
    def __init__(self, tokens): (...)
    def consumir(self, tipo_esperado): (...)
    def parse_programa(self): (...)
    def parse_instrucoes(self): (...)
    def parse_instrucao(self): (...)
    def parse_atribuicao_ou_entrada(self): (...)
    def parse_saida(self): (...)
    def parse_condicional(self): (...)
    def parse_loop(self): (...)
    def parse_condicao(self): (...)
    def parse_expressao(self): (...)
    def parse_termo(self): (...)
    def parse_fator(self): (...)
    def parse_retorno(self): (...)
```

```
def parse_fator(self):
    tipo, valor = self.token_atual
    if tipo == "NUMERO":
        self.consumir("NUMERO")
        return NoAST("NUMERO", valor=valor)
    elif tipo == "NOME":
        self.consumir("NOME")
        return NoAST("IDENTIFICADOR", valor=valor)
    elif tipo == "TEXTO":
        self.consumir("TEXTO")
        return NoAST("TEXTO", valor=valor)
    elif tipo == "ABRE_PAR":
        self.consumir("ABRE_PAR")
        exp = self.parse_expressao()
        self.consumir("FECHA_PAR")
    return exp
```



Gerador

```
def transpilador_python(no, nivel_indentacao=0):  
    //...  
    if no.tipo == "PROGRAMA":  
        codigo_py = "# Código gerado via Transpilador MineonScript\n\n"  
        for filho in no.filhos:  
            codigo_py += transpilador_python(filho, nivel_indentacao)  
        return codigo_py  
  
    elif no.tipo == "INSTRUcoes":  
        codigo_py = ""  
        for filho in no.filhos:  
            codigo_py += transpilador_python(filho, nivel_indentacao)  
        return codigo_py  
  
    elif no.tipo == "ATRIBUICAO":  
        var_nome = no.filhos[0].valor  
        exp = transpilador_python(no.filhos[1])  
        return f"{indent}{var_nome} = {exp}\n"
```

Exercício 1

Crie um programa que lê um valor e exiba seu dobro caso seja positivo, caso contrário exiba “VALOR INVÁLIDO”.

Faça o programa acima em *MineonScript*, ou na linguagem de transpilação que desenvolvestes, e então transpile-o para a linguagem Python.

Exercício 2

Altere o transpilador para gerar um código em uma outra linguagem à tua escolha.

TESTE



<https://docs.google.com/forms/d/e/1FAIpQLSfTlz5AWF3sxsC2SdowCik4N7eZpFNV5LVEVkHzH9ab8mrp0OA/viewform>

MineonScript - Versão final

```
import re # regular expressions
import sys # Sistema

# Definição dos tipos de tokens e seus padrões.
TOKENS = [
    ("COMENTARIO", r"//.*"),
    ("ESPACO", r"\s+"),
    ("SE", r"\bse\b"),
    ("SENAO", r"\bsenao\b"),
    ("ENQUANTO", r"\benquanto\b"),
    ("ESCREVA", r"\bescreva\b"),
    ("LEIA", r"\bleia\b"),
    ("RETORNO", r"\bretorna\b"),
    ("IGUALDADE", r"=="),
    ("IGUAL", r "="),
    ("MAIOR", r ">"),
    ("MENOR", r "<"),
    ("NEGACAO", r "!"),
    ("SOMA", r "+"),
    ("SUB", r "-"),
    ("MUL", r "\*"),
    ("DIV", r "/"),
    ("ABRE_CHAVE", r "{"),
    ("FECHA_CHAVE", r "}"),
    ("ABRE_PAR", r "("),
    ("FECHA_PAR", r ")"),
    ("NUMERO", r "\d+"),
    ("TEXTO", r "\".*?\\"",),
    ("NOME", r "[a-zA-Z_][a-zA-Z0-9_]*"),
```

TESTE

Qual é a função primária do Lexer (Analisador Léxico) em um compilador ou transpilador?

Ler a sequência de caracteres do código-fonte, ignorar ruídos como comentários e espaços, e agrupá-los em uma lista de tokens categorizados.

Validar a ordem gramatical das instruções e construir a Árvore Sintática Abstrata (AST) do programa.

Checar se os tipos de dados das variáveis são compatíveis com as operações executadas no código.

Converter a Árvore Sintática Abstrata (AST) diretamente em código executável ou em outra linguagem alvo.

Mapear o escopo e validar se as variáveis utilizadas foram previamente declaradas no programa.

O que caracteriza o papel do Parser (Analisador Sintático) no fluxo de processamento de linguagens?

Analisar a estrutura e a hierarquia da lista de tokens enviada pelo Lexer para construir a Árvore Sintática Abstrata (AST).

Converter o texto do código-fonte diretamente em instruções de linguagem de máquina ou bytecode.

Executar o código-fonte linha por linha e exibir a saída no terminal.

Garantir que não existam erros de lógica ou operações inválidas como divisão por zero no programa.

Substituir os nomes das variáveis por endereços de memória físicos antes da